

Data-driven Opportunity Constrained Multiple-Choice Knapsack Problem

Abstract: The Multi-Choice Knapsack Problem (MCKP) is a classic NP-hard combinatorial optimization problem. In real-world scenarios, it often arises in the form of the Multi-Choice Knapsack Problem with Chance Constraints (CCMCKP), where item weights are modeled as random variables. This paper proposes a solution approach for CCMCKP by transforming stochastic chance constraints into deterministic risk-averse constraints and solving the problem using a Genetic Algorithm. In addition, two solution methods are introduced: the RADP method, based on dynamic programming, and the RAGA method, which combines genetic algorithms and dynamic programming. While RADP guarantees an exact solution, it is primarily suitable for small-scale problems, as it struggles to handle large-scale instances efficiently. On the other hand, RAGA is an approximate solution method that excels in solving large-scale problems, offering scalability without sacrificing feasibility. Experimental results confirm the effectiveness of the proposed methods. For small-scale instances, both RADP and RAGA achieve optimal solutions, while for larger instances, RAGA demonstrates significantly higher efficiency compared to RADP, consistently delivering feasible solutions within a reasonable time frame. Moreover, when compared to directly applying a basic genetic algorithm to solve CCMCKP, RAGA outperforms in both solution quality and computational efficiency. These findings highlight the practical superiority of RAGA for large-scale applications, making it a valuable tool for real-world problem-solving.

Key words: data-driven optimization; combinatorial optimization; evolutionary algorithm

Contents

1 Introduction	1
2 Notations And Problem Definition	5
2.1 Problem Modeling	5
2.2 Transforming Chance Constraints Into Deterministic Risk-Averse Constraints	6
3 Solving CCMCKP	8
3.1 Solving CCMCKP Using Genetic Algorithm	8
3.2 Overview Of RAMCKP	10
3.3 Dynamic Programming-Based RAGA	13
3.4 Genetic Algorithm-Based RAGA	14
3.5 Solving CCMCKP Based On RAMCKP	15
4 Experiment And Result Analysis	25
4.1 Test Case Set	25
4.2 Performance Metrics	25
4.3 Experimental Results And Analysis	26
5 Conclusion	32
References	34

1 Introduction

The Multiple-Choice Knapsack Problem (MCKP) is a classic NP-hard combinatorial optimization problem with widespread applications across various critical domains, including portfolio selection, component selection for information technology systems, and supply chain optimization^[1]. Specifically, the MCKP consists of several disjoint item sets, where each set contains multiple items. Each item has two attributes: weight and cost. The goal of the MCKP is to select exactly one item from each set, subject to a predefined weight constraint (i.e., the knapsack constraint), in order to minimize the total cost of the selected items. Over the past few decades, the academic community has proposed various methods to solve the MCKP, including dynamic programming algorithms^[2], heuristic searches^[3], greedy methods^[4], and branch-and-bound techniques^[5].

Although significant progress has been made in the research on MCKP, existing solutions still struggle when the item weights are random variables rather than fixed values. This is because, in such cases, the constraints are no longer simply that the total weight should not exceed a fixed limit. Instead, the problem requires that the probability of the total weight exceeding the limit must be greater than or equal to a predefined threshold. Problems of this nature are referred to as the Chance-Constrained Multiple-Choice Knapsack Problem^[6] (CCMCKP).

The Chance-Constrained Multiple-Choice Knapsack Problem (CCMCKP) has a wide range of applications in several fields. A typical example is the end-to-end network configuration in 5G systems^[7]. In 5G commercial applications, user-plane latency is one of the

key performance indicators for low-latency communication. As shown in Fig. 1-1, the complete data packet transmission path in a 5G network consists of multiple parts, including the eNB, backhaul links, and the core network. These parts correspond to item sets in the CCMCKP, where each set has different implementation options (i.e., items), and only one option can be chosen from each set. The total transmission delay for the data packets is the cumulative delay caused by all the selected options. In this context, the packet transmission delay (analogous to the weight of an item) is a random variable, influenced by environmental factors or equipment failures, and thus, its value is not fixed. The objective of the 5G end-to-end network configuration is to choose one configuration option for each node in the network, ensuring that the probability of the transmission delay exceeding the maximum allowed delay does not fall below a specified threshold, while simultaneously minimizing the overall network configuration cost.

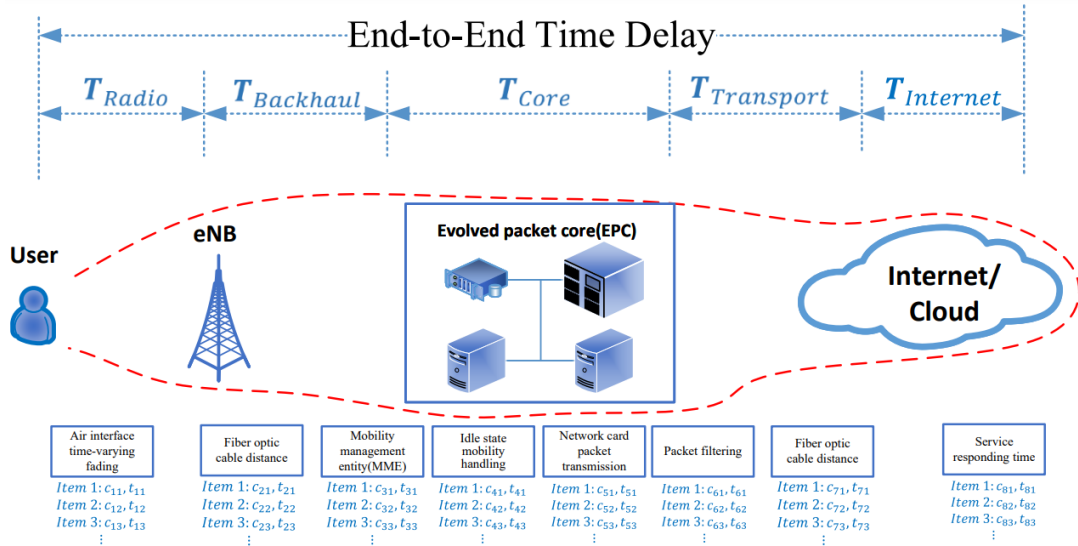


Fig. 1-1 5G end-to-end network configuration

Overall, the CCMCKP (Chance-Constrained Multiple-Choice Knapsack Problem) faces two main challenges: first, it is inherently an NP-hard problem, and second, it involves handling

chance constraints that contain random variables. Since the distribution of these random variables is unknown, Monte Carlo simulation can be used to estimate the distribution of the chance constraint. However, this method has limited accuracy and is computationally slow. To address this, the paper adopts a data-driven approach, leveraging the Central Limit Theorem. By analyzing large datasets, the distribution of the data can be inferred, and it can be shown that the distribution of the sample mean approaches a normal distribution, even if the original data does not follow a normal distribution^[8].

When the distribution conforms to a normal distribution, the chance constraint can be transformed into a deterministic risk-averse constraint. This allows the problem to be solved using a Genetic Algorithm (GA), a type of metaheuristic optimization algorithm. GA mimics natural selection processes, such as crossover and mutation, to evolve high-quality solutions. This method has demonstrated remarkable performance in solving various combinatorial optimization problems.

An alternative approach is to transform the CCMCKP into a series of risk-averse multiple-choice knapsack problems^[8] (RAMCKP). In this case, when the weights of the items follow a normal distribution, it can be proven that at least one optimal solution of the transformed RAMCKP matches the optimal solution of the original problem. To solve the RAMCKP, this paper introduces a complete solution method based on Dynamic Programming (DP), called RADP. RADP is an exact algorithm with optimality guarantees, but its scalability is limited, making it challenging to solve large-scale problems (e.g., where the number of item categories is ≥ 10 and the number of items per category is ≥ 10) within acceptable time limits.

Additionally, this paper presents a method that combines Genetic Algorithms with RAMCKP, referred to as RAGA. This method can solve large-scale problems, though the solutions obtained may not necessarily be optimal. Further analysis comparing the results of solving the CCMCKP using RAGA versus directly applying GA shows that the RAGA method yields better solutions than directly using GA to solve CCMCKP.

The structure of the paper is as follows: Section 2 provides the formal definition of CCMCKP; Section 3 introduces the methods for solving CCMCKP and offers detailed explanations; Section 4 describes the data samples in detail and presents the experimental results along with their analysis; and finally, Section 5 summarizes the findings of this research.

2 Notations And Problem Definition

2.1 Problem Modeling

Given m disjoint sets, denoted as $\{1, 2, \dots, m\}$, each set contains a number of factors. Without loss of generality, assume that each set contains the same number of factors, i.e., $N_1 = N_2 = \dots = N_m = N$. Each factor $j \in N_i$ has two attributes: cost c_{ij} and weight w_{ij} , where c_{ij} is a fixed value, and w_{ij} is a random variable. By applying the Central Limit Theorem (CLT) in a data-driven approach, the distribution of the weight can be inferred from a large dataset. This leads to the assumption that the weight follows a normal distribution, i.e., $w_{ij} \sim \mathcal{N}(\mu_{ij}, \sigma_{ij}^2)$, where μ_{ij} and σ_{ij}^2 represent the mean and variance of the normal distribution, respectively.

The objective of the Chance-Constrained Multiple-Choice Knapsack Problem (CCMCKP) is to minimize the total cost while satisfying a chance constraint on the total weight. Specifically, the total weight of the selected items should be less than W with a confidence level of at least P_0 , and only one factor can be chosen from each set.

Thus, the CCMCKP model is formulated as follows:

$$\min \sum_{i=1}^m \sum_{j \in N_i} c_{ij} x_{ij} \quad (2-1)$$

$$\text{s.t.} \begin{cases} P \left(\sum_{i=1}^m \sum_{j \in N_i} w_{ij} x_{ij} \leq W \right) \geq P_0 \\ \sum_{j \in N_i} x_{ij} = 1 ; i = 1, 2, \dots, m \\ x_{ij} \in \{0, 1\} ; i = 1, 2, \dots, m, j \in N_i \end{cases} \quad (2-2)$$

$$\sum_{i=1}^m \min_{j \in N_i, 1 \leq l \leq L} d_{ijl} \leq W \leq \sum_{i=1}^m \max_{j \in N_i, 1 \leq l \leq L} d_{ijl} \quad (2-3)$$

For each factor, we collect L sample data points, denoted as d_{ijl} . To avoid trivial solutions, we also assume that the inequality in equation (2-3) holds.

2.2 Transforming Chance Constraints Into Deterministic Risk-Averse Constraints

Due to the occurrence of the chance constraint, as shown in equation (2-4), within the constraint (2-2), we need to transform equation^[9] (2-4). Since the random variables w_{ij} follow independent normal distributions, and the sum of independent normal distributions is also normally distributed, we can derive equation (2-5).

$$P\left(\sum_{i=1}^m \sum_{j \in N_i} w_{ij} x_{ij} \leq W\right) \geq P_0 \quad (2-4)$$

$$\sum_{i=1}^m \sum_{j \in N_i} w_{ij} x_{ij} \sim \mathcal{N}\left(\sum_{i=1}^m \sum_{j \in N_i} \mu_{ij} x_{ij}, \sum_{i=1}^m \sum_{j \in N_i} \sigma_{ij}^2 x_{ij}\right) \quad (2-5)$$

Thus, the chance constraint in equation (2-4) can be transformed as follows:

$$\begin{aligned} & P\left(\sum_{i=1}^m \sum_{j \in N_i} w_{ij} x_{ij} \leq W\right) \geq P_0 \\ \Leftrightarrow & P\left(\frac{\sum_{i=1}^m \sum_{j \in N_i} w_{ij} x_{ij} - \sum_{i=1}^m \sum_{j \in N_i} \mu_{ij} x_{ij}}{\sqrt{\sum_{i=1}^m \sum_{j \in N_i} \sigma_{ij}^2 x_{ij}}} \leq \frac{W - \sum_{i=1}^m \sum_{j \in N_i} \mu_{ij} x_{ij}}{\sqrt{\sum_{i=1}^m \sum_{j \in N_i} \sigma_{ij}^2 x_{ij}}}\right) \geq P_0 \\ \Leftrightarrow & \Phi\left(\frac{W - \sum_{i=1}^m \sum_{j \in N_i} \mu_{ij} x_{ij}}{\sqrt{\sum_{i=1}^m \sum_{j \in N_i} \sigma_{ij}^2 x_{ij}}}\right) \geq P_0 \\ \Leftrightarrow & \frac{W - \sum_{i=1}^m \sum_{j \in N_i} \mu_{ij} x_{ij}}{\sqrt{\sum_{i=1}^m \sum_{j \in N_i} \sigma_{ij}^2 x_{ij}}} \geq \Phi^{-1}(P_0) \\ \Leftrightarrow & \sum_{i=1}^m \sum_{j \in N_i} \mu_{ij} x_{ij} + \Phi^{-1}(P_0) \sqrt{\sum_{i=1}^m \sum_{j \in N_i} \sigma_{ij}^2 x_{ij}} \leq W \end{aligned}$$

By replacing the original chance constraint (2-4) with this deterministic risk-averse constraint, we obtain the equivalent CCMCKP model:

$$\min \sum_{i=1}^m \sum_{j \in N_i} c_{ij} x_{ij} \quad (2-6)$$

$$\text{s.t.} \begin{cases} \sum_{i=1}^m \sum_{j \in N_i} \mu_{ij} x_{ij} + \Phi^{-1}(P_0) \sqrt{\sum_{i=1}^m \sum_{j \in N_i} \sigma_{ij}^2 x_{ij}} \leq W \\ \sum_{j \in N_i} x_{ij} = 1 ; i = 1, 2, \dots, m \\ x_{ij} \in \{0, 1\} ; i = 1, 2, \dots, m, j \in N_i \end{cases} \quad (2-7)$$

3 Solving CCMCKP

Since the constraint in equation (2-7) still includes nonlinear terms, directly solving the problem is challenging. As a result, this paper proposes three distinct solution approaches. The first method applies a genetic algorithm to directly solve the CCMCKP model with deterministic risk-averse constraints. The other two methods convert the CCMCKP into a series of iterative RAMCKP models, which are then solved using dynamic programming and genetic algorithms, respectively. Finally, the paper provides a computational complexity analysis for each of these approaches.

3.1 Solving CCMCKP Using Genetic Algorithm

The Genetic Algorithm (GA) is an optimization method inspired by the principles of natural selection and genetics, widely used to solve complex combinatorial optimization problems. When applied to solve the CCMCKP, the genetic algorithm simulates biological evolutionary processes such as selection, crossover, and mutation. These mechanisms enable it to effectively explore the solution space and find near-optimal solutions^[10].

Algorithm 1: Genetic Algorithm

Input: The cost and weight of each item, knapsack's capacity W , confidence level P_0 and maximum number of iterations t_{max} .

Output: the optimal feasible solution S_{opt} .

- 1: Initialize the population $\mathcal{P}$, $i = 0$
 - 2: Repair infeasible solutions
 - 3: **while** $i < t_{max}$ **do**
 - 4: Dominant population selection
 - 5: Crossover and mutation
 - 6: Repair infeasible solutions
 - 7: Update the population $\mathcal{P}$
 - 8: $i \leftarrow i + 1$
 - 9: **end while**
-

10: $S_{opt} \leftarrow$ the optimal solution of the population $\mathcal{P}$

11: **return** S_{opt}

The genetic algorithm for solving the CCMCKP (Constrained Capacitated Multiple Choice Knapsack Problem) is outlined in Algorithm 1, as detailed below:

1. Random Initialization of Population: The population $\mathcal{P}$, containing K solutions, is randomly initialized. K solutions are generated, and if any of them are infeasible, they undergo a repair process.

2. Fitness Calculation: The fitness of each individual solution is calculated. The fitness is inversely proportional to the cost of the solution.

3. Elite Selection: Select e elite individuals from the population. The individuals are sorted based on fitness in descending order, and the top e individuals are selected to form the elite population. This ensures that the best individuals of each generation are retained, thereby improving the quality of the final output.

4. Parent Selection: The probability of selection for parent individuals is determined based on their fitness in the elite population. Two parents are chosen randomly without replacement, with the probability of selection proportional to their fitness.

5. Crossover and Mutation: A crossover operation is performed on each individual in the selected parents with a crossover probability p_c . Afterward, the offspring undergo random mutation, where the mutation probability is set to $p_m = 1/m$.

6. Repair of Infeasible Offspring: If the offspring generated is infeasible, it undergoes a repair process.

7. Repeat Steps 4–6: Steps 4 to 6 are repeated $e/2$ times to generate sufficient offspring.

8. Population Update: All offspring are added to the population $\mathcal{P}$, and individuals with the lowest fitness are eliminated. The population is then reduced to retain only K individuals.

9. Iterative Process: Steps 3 to 8 are repeated for t_{max} generations. After the algorithm ends, the solution with the lowest cost, S_{opt} , is returned.

In the solution repair step, during the algorithm's execution, each generated solution is first evaluated using an evaluation function. If the current solution does not meet the deterministic risk-averse constraints, it enters the repair process. In this repair process, the weight w_{ij} of each item in the class is computed, and the item with the largest weight in the current solution is selected. The next item in the class that has a smaller weight than the currently selected item is identified, and the item corresponding to the selected class in the current solution is replaced by this new item. After completing the above steps, the solution is re-evaluated, and the process is repeated until the solution satisfies the confidence requirements.

The time complexity for evaluating each solution is $O(1)$, so the time required for repairing a solution is $O(mN)$. The time complexity for population initialization is $O(KmN)$. The total number of iterations required for population evolution is $O(K^2 t_{max} mN)$. It should be noted that as the proportion of feasible solutions in the population increases, the number of search iterations required during the repair process decreases. This means that the time required for a single repair is much less than $O(mN)$, resulting in an overall reduction in the algorithm's runtime.

3.2 Overview Of RAMCKP

The RAMCKP (Risk-Aware Multiple Choice Knapsack Problem) is derived from the

CCMCKP (Constrained Capacitated Multiple Choice Knapsack Problem) by transforming its nonlinear constraints into linear ones. The model of RAMCKP is as follows:

$$\min \sum_{i=1}^m \sum_{j \in N_i} c_{ij} x_{ij} \quad (3-1)$$

$$\text{s.t.} \begin{cases} \sum_{i=1}^m \sum_{j \in N_i} (\mu_{ij} + \lambda \sigma_{ij}^2) x_{ij} \leq W' \\ \sum_{j \in N_i} x_{ij} = 1 ; i = 1, 2, \dots, m \\ x_{ij} \in \{0, 1\} ; i = 1, 2, \dots, m, j \in N_i \end{cases} \quad (3-2)$$

where λ is the risk-averse parameter that weighs the mean and variance of the weight of each item. The parameter W' represents the weight constraint in the current RAMCKP. It can be observed that when $\lambda = 0$ and $W = W'$, RAMCKP reduces to the MCKP (Multiple Choice Knapsack Problem) with fixed item weights μ_{ij} .

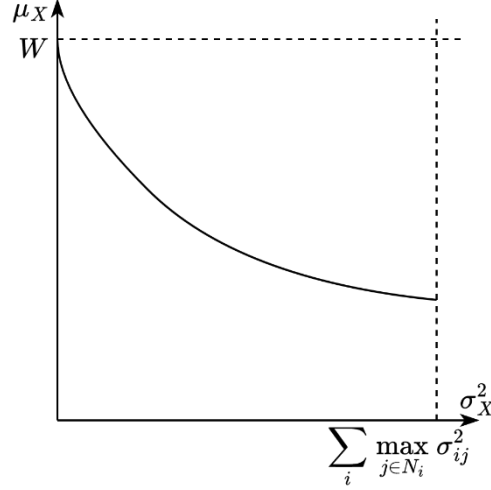


Fig. 3-1 Variance-mean plane

To better explain what RAMCKP is, we introduce an important concept—the “Variance-Mean Plane^[8]”. For each feasible solution X in the solution set of CCMCKP, let σ_X^2 denote the sum of the variances of the selected items corresponding to solution X , i.e. $\sigma_X^2 = \sum_{i=1}^m \sum_{j \in N_i} \sigma_{ij} x_{ij}$. and let μ_X represent the sum of the means of the selected items corresponding

to solution X , i.e. $\mu_x = \sum_{i=1}^m \sum_{j \in N_i} \mu_{ij} x_{ij}$. Thus, the combination of random variables for each solution X corresponds to a pair (σ_x^2, μ_x) . If the variance σ_x^2 is plotted on the horizontal axis and the mean μ on the vertical axis, we obtain the plane shown in Fig. 3-1.

In Fig. 3-1, the arcs correspond to the cases where the nonlinear constraints in CCMCKP (specifically constraints (2-7)) hold as equalities, as described by equation (3-3). The area enclosed by the arcs and the coordinate axes forms the feasible region of the CCMCKP solution.

$$\sum_{i=1}^m \sum_{j \in N_i} \mu_{ij} x_{ij} + \Phi^{-1}(P_0) \sqrt{\sum_{i=1}^m \sum_{j \in N_i} \sigma_{ij}^2 x_{ij}} = W \quad (3-3)$$

$$\sum_{i=1}^m \sum_{j \in N_i} (\mu_{ij} + \lambda \sigma_{ij}^2) x_{ij} = W' \quad (3-4)$$

Now, if we consider the case where the nonlinear constraints in RAMCKP's constraint (3-2) hold as equalities, as represented by equation (3-4), it corresponds to a straight line in the Variance-Mean plane, as shown in Fig. 3-2. Here, $-\lambda$ represents the slope of the line, and W' represents its intercept. The area enclosed by the line and the coordinate axes forms the feasible region of the RAMCKP solution.

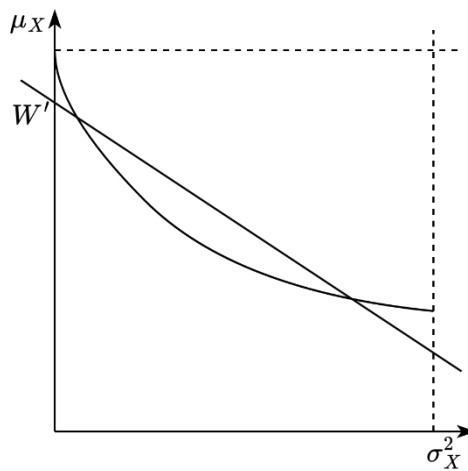


Fig. 3-2 The line corresponding to the constraint

Before discussing specific methods for solving CCMCKP through RAMCKP, we will first present two approaches for solving RAMCKP: the Dynamic Programming-based RADP (Risk-Aware Dynamic Programming) and the Genetic Algorithm-based RAGA (Risk-Aware Genetic Algorithm).

3.3 Dynamic Programming-Based RAGA

Dynamic Programming (DP) is a method for solving complex problems by breaking them down into simpler subproblems. Previous studies have shown that the RAMCKP can be solved using DP with polynomial time complexity. The specific process is as follows: Let $\omega_{ij} = \mu_{ij} + \lambda\sigma_{ij}^2$, where the goal is to solve for the optimal value of the MCKP when choosing items from the first u (where $(1 \leq u \leq m)$) categories, and with a knapsack capacity constraint of v (where $(0 \leq v \leq W')$). The objective is to minimize the cost given the constraints.

$$\min \sum_{i=1}^u \sum_{j \in N_i} c_{ij} x_{ij} \quad (3-5)$$

$$\text{s.t.} \begin{cases} \sum_{i=1}^u \sum_{j \in N_i} \omega_{ij} x_{ij} \leq v \\ \sum_{j \in N_i} x_{ij} = 1 ; i = 1, 2, \dots, u \\ x_{ij} \in \{0, 1\} ; i = 1, 2, \dots, u, j \in N_i \end{cases} \quad (3-6)$$

To solve this problem, we define the state $f(u, v)$ as the minimum cost required to select items from the first u categories such that their total weight does not exceed v . The state transition is governed by the items selected from the u -th category. The state transition equation is given by equation (3-7).

$$f(u, v) = \min_{j \in N_u, \omega_{uj} \leq v} f(u-1, v - \omega_{uj}) + c_{uj} \quad (3-7)$$

Finally, $f(m, W')$ represents the optimal solution to the RAMCKP. The specific steps of the RADP algorithm are shown in Algorithm 2.

The time complexity^[2] of the dynamic programming algorithm is $O(W' \sum_{i=1}^m N_i) = O(W'mN)$. When the number of items is large, W' usually increases as well, which causes the algorithm's complexity to rise sharply. Considering that solving the CCMCKP requires multiple iterations of solving a series of RAMCKPs, the overall time complexity becomes very high, and scalability is limited. Based on this, we propose using a genetic algorithm to efficiently solve the RAMCKP.

Algorithm 2: RADP

Input: The cost and weight of each item, knapsack's capacity W' .

Output: Minimum cost.

```

1: for  $v = 0$  to  $W'$  do
2:    $f(0, v) = 0$ 
3: end for
4: for  $u = 1$  to  $m$  do
5:   for  $v = 0$  to  $W'$  do
6:      $f(u, v) = +\infty$ 
7:     for  $j = 1$  to  $N_u$  do
8:       if  $\omega_{uj} \leq v$  then
9:          $f(u, v) = \min\{f(u, v), f(u-1, v - \omega_{uj}) + c_{uj}\}$ 
10:      end if
11:    end for
12:  end for
13: end for
14: return  $f(m, W')$ 

```

3.4 Genetic Algorithm-Based RAGA

By simply replacing the nonlinear constraints used in Algorithm 1 to handle the CCMCKP with the linear constraints of the RAMCKP, the algorithm can efficiently solve the RAMCKP.

The upper bound of the time complexity is also $O(K^2 t_{max} mN)$.

This method is highly effective for solving large-scale problems, especially in cases where dynamic programming becomes difficult to implement due to the size of the problem. It offers significant computational efficiency in these situations. However, since the genetic algorithm is a heuristic optimization method, it does not exhaustively search all possible solutions to find the optimal one. As a result, the solution obtained is not guaranteed to be the global optimum and may instead be a local optimum. This means that while the method performs exceptionally well on large-scale problems, it cannot guarantee the discovery of the optimal solution every time.

3.5 Solving CCMCKP Based On RAMCKP

The overall approach involves transforming the CCMCKP into a series of iterative RAMCKP solutions, and solving each RAMCKP using the previously mentioned RADP or RAGA algorithms. This process progressively approximates the optimal solution of the CCMCKP.

First, we fix W' in the RAMCKP as W , and initialize $k = 1$, $\lambda_k = 0$. Then, by continuously adjusting the value of λ_k and solving different RAMCKPs, we find a solution that simultaneously satisfies both the RAMCKP and Equation (3-8). This solution will be a feasible solution to the CCMCKP.

$$\sum_{i=1}^m \sum_{j \in N_i} \mu_{ij} x_{ij} + \Phi^{-1}(P_0) \sqrt{\sum_{i=1}^m \sum_{j \in N_i} \sigma_{ij}^2 x_{ij}} \leq W \quad (3-8)$$

The specific steps are as follows: First, solve the RAMCKP with parameters (λ_1, W) to obtain the solution X_1 . If X_1 satisfies Equation (3-8), then X_1 is the optimal solution to the

CCMCKP. This is because, in this case, the feasible solution space of the CCMCKP is a subset of the feasible solution space of the RAMCKP solution, as shown in Fig. 3-3. For any solution that satisfies the RAMCKP constraints, it must also satisfy the CCMCKP constraints, as illustrated in Fig. 3-4. Given that X_1 is the optimal solution to the RAMCKP, it will also be the optimal solution to the CCMCKP. Note that this result holds only for the first solution of the RAMCKP.

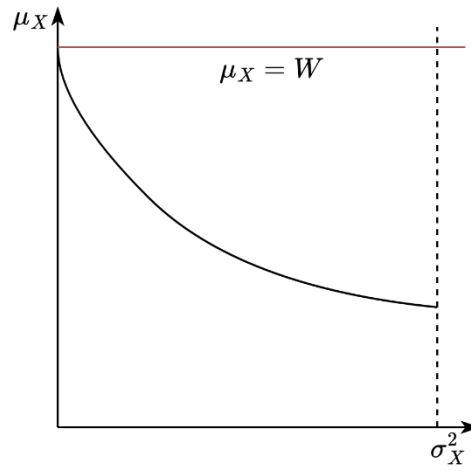


Fig. 3-3 The feasible space of CCMCKP is a subset of RAMCKP's

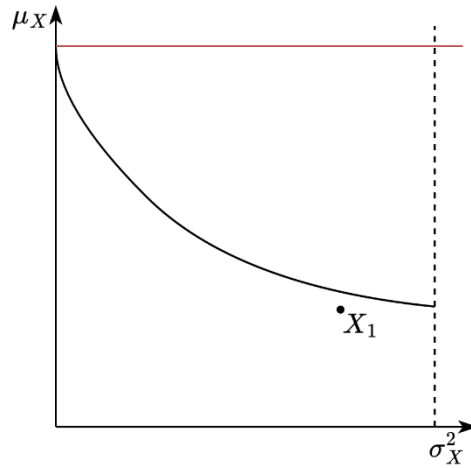


Fig. 3-4 The solution satisfies the constraints of CCMCKP

If X_k does not satisfy Equation (3-8), it means that X_k lies between the region formed by the arc represented by Equation (3-3) and the line represented by Equation (3-4), and thus is

not within the feasible region of the CCMCKP, as shown in Fig. 3-5. Since the intercept of the line is fixed at W , we need to decrease the slope of the line, $-\lambda_k$, in order to exclude X_k from the region formed by the line and the coordinate axes. This is to prevent obtaining X_k again in the next iteration when solving the RAMCKP. Therefore, the next iteration's $\lambda_{k+1} = -\frac{W-\mu_{X_k}}{\sigma_{X_k}^2} - \varepsilon$, where ε is a small positive number.

Since X_k does not satisfy the nonlinear constraint of the CCMCKP, we have $\mu_{X_k} + \Phi^{-1}(P_0) \cdot \sigma_{X_k} > W$, implying that $\Phi^{-1}(P_0) > \frac{W-\mu_{X_k}}{\sigma_{X_k}}$. This leads to $-\frac{W-\mu_{X_k}}{\sigma_{X_k}^2} = -\frac{1}{\sigma_{X_k}} \frac{W-\mu_{X_k}}{\sigma_{X_k}} > \frac{\Phi^{-1}(P_0)}{\sigma_{X_k}}$. Therefore, the next iteration's $\lambda_{k+1} = \frac{\Phi^{-1}(P_0)}{\sigma_{X_k}}$, as shown in Fig. 3-6.

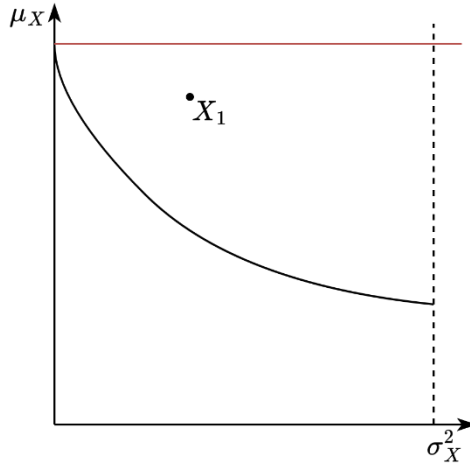


Fig. 3-5 The solution does not satisfy the constraints of CCMCKP

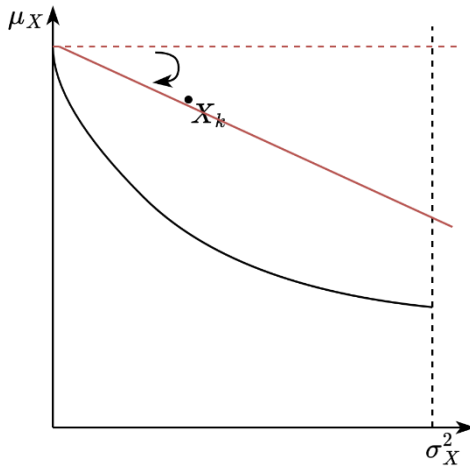


Fig. 3-6 Refine the line slope to exclude the previously obtained solution

This process is repeated until the k -th solution X_k obtained from solving the RAMCKP lies within the feasible region of the CCMCKP, as shown in Fig. 3-7. Such a solution must exist because the slope of the line continually decreases, ultimately reaching a situation where the feasible solution space of the RAMCKP is a subset of the feasible solution space of the CCMCKP.

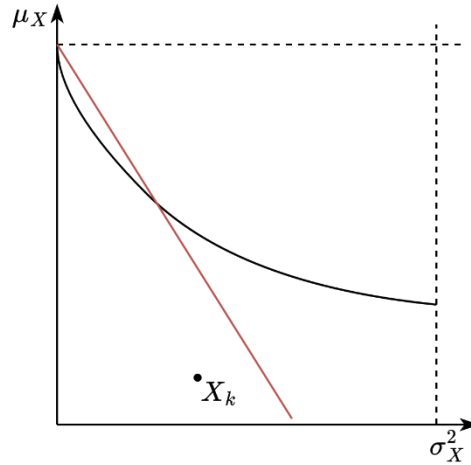


Fig. 3-7 Find a solution that satisfies the CCMCKP constraints through iteration

The solution X_k is added to the solution set $\mathcal{X}$. The solution X_k represents the optimal solution of the intersection of the feasible regions of the CCMCKP and RAMCKP. The next step involves solving for the optimal solution in another portion of the feasible region of CCMCKP.

$$\begin{cases} \mu_{X_k} + \Phi^{-1}(P_0)\sigma_{X_k} = W \\ \mu_{X_k} + \lambda_k\sigma_{X_k}^2 = W \end{cases} \quad (3-9)$$

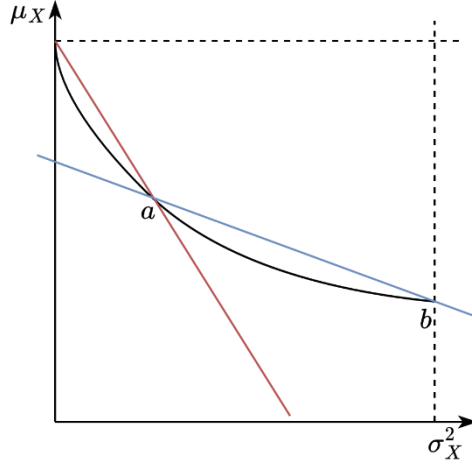


Fig. 3-8 The straight line formed by points a and b

By solving the system of equations involving the arc and the straight line (Equations 3-9), we can determine the intersection points of the arc and the straight line:

point $a \left(\frac{(\Phi^{-1}(P_0))^2}{\lambda_k^2}, W - \frac{(\Phi^{-1}(P_0))^2}{\lambda_k} \right)$, and point $b \left(\sum_i \max_{j \in N_i} \sigma_{ij}^2, W - \Phi^{-1}(P_0) \sqrt{\sum_{i=1}^m \max_{j \in N_i} \sigma_{ij}^2} \right)$.

The slope of the line formed by points a and b is denoted as λ_{ab} , and the intercept is W_{ab} , as shown in Fig. 3-8.

It can be seen that the feasible region of the RAMCKP with parameters (λ_{ab}, W_{ab}) encompasses the remaining portion of the CCMCKP's feasible region, which has not been explored. This allows us to solve for the RAMCKP with parameters (λ_{ab}, W_{ab}) . The pair (λ_{ab}, W_{ab}) is then added to the parameter set Λ .

When the parameter set Λ is non-empty, a parameter pair $(\hat{\lambda}, \hat{W})$ is selected from Λ and removed from the set. Subsequently, the RADP or RAGA algorithm is used to solve for the RAMCKP with parameters $(\hat{\lambda}, \hat{W})$. If the solution X satisfies Equation (3-8), as shown in Fig. 3-9, it is added to the solution set $\mathcal{X}$. Otherwise, as shown in Fig. 3-10, in order to continue searching the remaining portion of the feasible region, two straight lines need to be constructed

with X as an endpoint, each intersecting points a and b (the two intersection points of the arc and the straight line), as shown in Fig. 3-11.

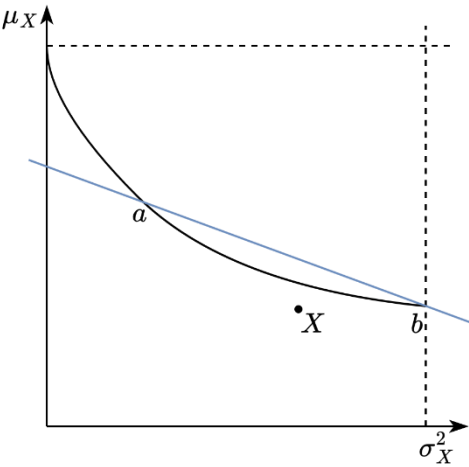


Fig. 3-9 The solution satisfies the constraints of CCMCKP

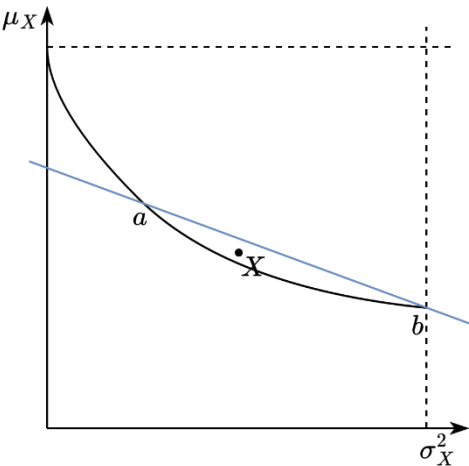


Fig. 3-10 The solution does not satisfy the constraints of CCMCKP

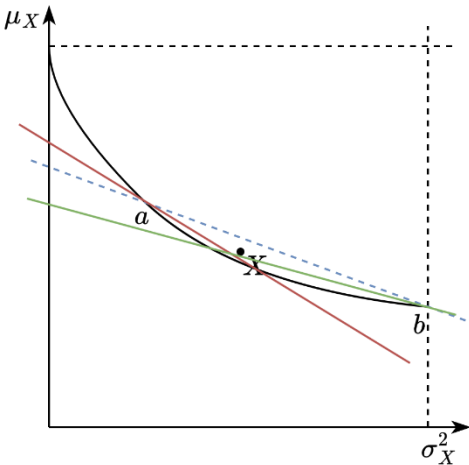


Fig. 3-11 Construct two new lines

The line constructed with point a has a slope and intercept given by: $\widehat{\lambda}_a = \frac{\mu_x - \mu_a}{\sigma_x^2 - \sigma_a^2} - \varepsilon$, $\widehat{W}_a = \mu_b - \widehat{\lambda}_a \sigma_b^2$. while the line constructed with point b has a slope and intercept given by: $\widehat{\lambda}_b = \frac{\mu_x - \mu_b}{\sigma_x^2 - \sigma_b^2} + \varepsilon$, $\widehat{W}_b = \mu_b - \widehat{\lambda}_b \sigma_b^2$. The pairs $(\widehat{\lambda}_a, \widehat{W}_a)$ and $(\widehat{\lambda}_b, \widehat{W}_b)$ are added to the parameter set Λ . The above steps are repeated until the parameter set Λ becomes empty.

Finally, the solution with the lowest cost from the solution set $\mathcal{X}$, denoted as S_{opt} , is selected as the optimal solution, and the algorithm ends.

The entire algorithm flow for solving the CCMCKP using the RAMCKP is shown in Algorithm 3.

Algorithm 3: RAMCKP-Based CCMCKP Solver

Input: The cost and weight of each item, knapsack's capacity W , confidence level P_0 .

Output: the optimal feasible solution S_{opt} .

- 1: Initialize $k = 1$, $\lambda_k = 0$, $\mathcal{X} = \emptyset$
 - 2: Solve RAMCKP(λ_k, W) using RADP or RAGA to obtain the solution X_k
 - 3: **while** X_k is not a feasible solution to CCMCKP **do**
 - 4: $\lambda_{k+1} = \Phi^{-1}(P_0)/\sigma_{X_k}$
 - 5: $k = k + 1$
 - 6: Solve RAMCKP(λ_k, W) using RADP or RAGA to obtain the solution X_k
 - 7: **end while**
 - 8: $\mathcal{X} \leftarrow \{X_k\}$
 - 9: **if** $k = 1$ **then**
 - 10: $S_{opt} = \min(\mathcal{X})$
 - 11: **return** S_{opt}
 - 12: **end if**
 - 13: Calculate the intersection point a of the line and arc, and fixed boundary point b .
 - 14: Calculate the slope λ_{ab} and intercept W_{ab} of the line through a and b
 - 15: Initialize $\Lambda = \emptyset$
 - 16: $\Lambda \leftarrow (\lambda_{ab}, W_{ab})$
 - 17: **while** Λ is not empty **do**
 - 18: $(\widehat{\lambda}, \widehat{W}) \in \Lambda$
 - 19: $\Lambda \setminus \{(\widehat{\lambda}, \widehat{W})\}$
 - 20: Solve RAMCKP($\widehat{\lambda}, \widehat{W}$) using RADP or RAGA to obtain the solution X
-

```

21:   if  $X$  is a feasible solution to CCMCKP then
22:        $\mathcal{X} \leftarrow \{X\}$ 
23:   else
24:       Calculate the intersection point  $a$  and  $b$  of the line and arc
25:       Calculate the slope  $\hat{\lambda}_1$  and intercept  $\hat{W}_1$  of the line through  $a$  and  $X$ 
26:       Calculate the slope  $\hat{\lambda}_2$  and intercept  $\hat{W}_2$  of the line through  $b$  and  $X$ 
27:        $\Lambda \leftarrow (\hat{\lambda}_1, \hat{W}_1), \Lambda \leftarrow (\hat{\lambda}_2, \hat{W}_2)$ 
28:   end if
29: end while
30:  $S_{opt} = \min(\mathcal{X})$ 
31: return  $S_{opt}$ 

```

The following is the theoretical performance analysis of solving the CCMCKP based on the RAMCKP.

The optimal solution of RAMCKP with chance constraints serves as an upper bound for the optimal solution of CCMCKP. To ensure that the optimal solution of RAMCKP satisfies the chance constraints, the corresponding point on the variance-mean plane must lie within the intersection of the feasible regions of both CCMCKP and RAMCKP. If the optimal solution of CCMCKP falls within this intersection, the optimal objective values of both CCMCKP and RAMCKP are identical. However, if the solution does not lie within this intersection, it indicates that the optimal objective value of RAMCKP exceeds that of CCMCKP. Therefore, the optimal objective value of RAMCKP is always greater than or equal to the optimal objective value of CCMCKP.

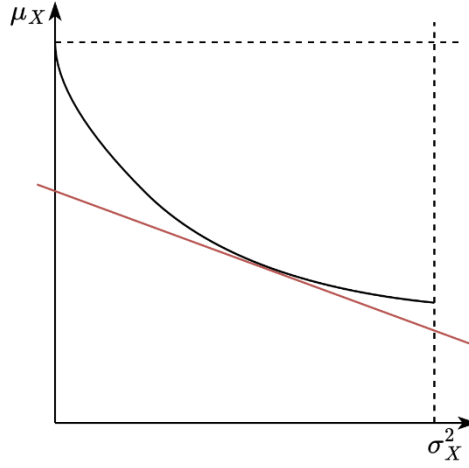


Fig. 3-12 The case of tangents

If a pair (λ, W') corresponds to a risk-averse constraint that is tangent to the parabola representing the chance constraints, as shown in Fig. 3-12, then the optimal solution of this RAMCKP is guaranteed to be a feasible solution to the original CCMCKP. This is because when the linear constraint of RAMCKP is tangent to the parabola, the feasible region of RAMCKP becomes a subset of the feasible region of CCMCKP. As a result, all points corresponding to feasible solutions, including the optimal solution of RAMCKP, must lie within the feasible region of CCMCKP. This leads to the conclusion that there exists at least one RAMCKP whose optimal solution is also the optimal solution of the original CCMCKP.

Based on the above analysis, when the RADP stops, the optimal solution in the solution set will correspond to the global optimal solution of CCMCKP. If the running time of RAGA is sufficient, upon termination, the optimal solution in the solution set will also represent the global optimal solution of CCMCKP. For RADP, when the optimal solution of RAMCKP lies within the intersection of the feasible regions of both the chance constraints and the risk-averse constraints, it becomes the local optimal solution within this intersection. Upon termination, the feasible region of CCMCKP has been fully explored, and therefore, the solution set contains all

local optimal solutions, making the optimal solution the global optimal solution of CCMCKP.

For IGA, since its mutation probability is greater than zero, any solution in the search space has a non-zero probability of being sampled. As the running time increases, the probability of finding the optimal solution of RAMCKP approaches one. Consequently, RAGA is also able to completely explore the feasible region of CCMCKP, and the final optimal solution in the solution set is the global optimal solution of CCMCKP.

4 Experiment And Result Analysis

This study constructs test cases of varying scale and constraint intensity, and based on these cases, conducts a performance comparison of genetic algorithms in solving CCMCKP, RADP, and RAIGA.

4.1 Test Case Set

The test case set used in this study is derived from existing problem examples^[8]. In each case, the weight of each item is treated as a Gaussian random variable. The mean and variance of this random variable are generated from a uniform distribution, where the mean $\mu_{ij} \sim \mathcal{U}(800, 3200)$ and the variance $\sigma_{ij}^2 \sim \mathcal{U}(1200, 22800)$. The smallest problem scale is set with $m = 3$ and $N = 3$, while the largest scale is $m = 100$ and $N = 50$. Depending on the values of m and N , the problem scale is categorized into small, medium, and large. The probability threshold for the chance constraints is uniformly set to $P_0 = 0.99$.

To assess the algorithm's performance under different constraint intensities, the minimum and maximum values (W_{min} and W_{max}) that satisfy the confidence requirement of P_0 are first calculated for each test case. Then, the value of W is determined according to the formula $(W_{max} - W_{min}) \times q + W_{min}$, where q is used to control the constraint intensity. The value of q satisfies the condition $0 < q < 1$, with smaller values of q corresponding to stronger constraint intensities. The cost of each item follows a uniform distribution, $c_{ij} \sim \mathcal{U}(50, 140)$.

4.2 Performance Metrics

This study uses three key performance metrics to assess the effectiveness of the algorithms:

1. Solving Time (Runtime): This metric measures the average time taken by the algorithm to find the optimal solution, representing the efficiency of the solution process. A smaller value indicates better performance. The algorithms being tested are constrained to a maximum runtime of 1800 seconds. If an algorithm successfully finds the optimal solution within the allocated time, the run is considered a "success", and the corresponding time t is recorded. If the algorithm fails to find the solution within the time limit, it is marked as a "timeout". Additionally, if the algorithm is interrupted due to timeout or an overload caused by exceeding memory limits, the result is recorded as “—”.

2. Cost of the Optimal Solution (Cost): This metric evaluates the quality of the optimal solution, with smaller values being preferable. It reflects how effectively the algorithm is able to minimize the cost of the selected items while adhering to the constraints.

RA Times: This metric refers to the number of RAMCKP instances solved by the algorithm during its execution. A smaller value signifies that the algorithm exhibits better exploration capacity, meaning it is able to identify high-quality solutions with fewer instances of the problem being solved.

4.3 Experimental Results And Analysis

The results of the algorithm on the test sample sets are presented in Tab. 1 and Tab. 2, with all data representing the "mean (standard deviation)" statistics obtained from 40 independent experiments. The samples in the table are categorized by scale, with "W" indicating different weight constraints. "GA" refers to the genetic algorithm applied directly to solve CCMCKP, "RADP" represents the dynamic programming algorithm based on RAMCKP, and "RAGA"

refers to the genetic algorithm based on RAMCKP. "Cost" represents the optimal solution's cost, "Runtime" denotes the actual running time of the algorithm, and "RA times" refers to the number of times RAMCKP was solved.

The experimental results indicate that as the problem scale increases, the running time of all algorithms significantly rises. Notably, RADP fails to produce results for medium-scale problems due to excessive runtime or memory overload, which causes the algorithm to terminate without completing. For problems of the same scale, as "W" decreases (i.e., the intensity of the opportunity constraint increases), the running time of the algorithms also increases. This is because the search for feasible solutions becomes more difficult. Furthermore, for medium-scale problems, the runtime of RADP grows exponentially as "N" increases, while GA and RAGA do not exhibit this behavior. Especially for larger medium-scale problems, the running times of GA and RAGA are much lower than that of RADP. Moreover, the runtime difference between GA and RAGA for samples with the same number of classes is relatively small, indicating that they are less sensitive to the number of factors "N" and are more influenced by "W".

For small-scale problems, the experimental results show that RADP can obtain the global optimal solution, while RAGA can achieve results almost identical to RADP. In solving medium-scale problems, RAGA provides solutions comparable to RADP for the cases solvable by RADP. However, for large-scale samples, RADP cannot directly solve the problem, while RAGA can effectively obtain feasible solutions. Regardless of the problem scale, the optimal solution found by GA is inferior to that found by RAGA.

As a result, with the increasing problem scale, RAGA's advantage in running efficiency gradually becomes apparent. It can obtain better solutions at a much lower time cost, even for problem sizes that RADP cannot handle. Additionally, although RAGA has a slightly longer runtime than GA, its solution quality is consistently superior to that of GA.

Further observation of the "RA times" results reveals that as the problem scale grows, RAGA maintains a high exploration efficiency, requiring only a small number of RAMCKP instances to thoroughly explore the solution space. This demonstrates the high efficiency of the proposed solution framework.

In conclusion, RAGA proves to be a highly efficient algorithm that excels in handling larger-scale problems and provides superior solutions compared to both RADP and GA. Its ability to maintain efficiency even with the increase in problem size makes it an ideal choice for practical applications involving large-scale optimization tasks.

Tab. 1 Comparisons of GA, RA-DP and RAGA on low constraint strength of CCMCKP instances

Samples		W	GA		RADP			RAGA		
			Cost	Runtime	Cost	Runtime	RA times	Cost	Runtime	RA times
Small scale	m=3 N=3	6511.6	226.213	1.921	225.402	0.007	1.0	225.402	1.582	1.0
	m=3 N=5	5404.8	244.812	1.849	244.001	0.015	3.0	244.001	4.621	3.0
	m=3 N=10	4255.6	212.172	2.134	211.919	0.021	3.0	211.919	5.821	3.0
	m=3 N=20	4457.0	282.195	2.751	281.237	0.027	3.0	281.262	7.531	3.0
	m=5 N=3	8875.6	490.852	1.975	488.237	0.021	3.0	488.237	5.426	3.0
	m=5 N=5	8455.2	369.241	2.412	367.127	0.025	4.0	367.127	7.291	4.0

	m=5 N=10	7742.4	317.256	2.317	316.726	0.031	3.0	316.726	6.411	3.0
	m=5 N=20	7137.8	303.827	2.532	300.921	0.049	3.0	303.101	8.342	3.3
Medium scale	m=10 N=3	20135.8	866.921	5.182	866.684	0.032	3.0	866.684	14.571	3.0
	m=10 N=5	17953.8	717.231	5.391	716.727	0.165	3.0	716.921	14.912	3.1
	m=10 N=10	15133.0	699.427	6.384	698.238	3.947	3.0	698.238	19.286	3.0
	m=10 N=20	14023.2	583.461	7.154	581.904	3.427	3.0	582.462	22.301	3.2
	m=20 N=3	35975.4	1478.134	7.602	1474.784	2.217	4.0	1477.826	21.928	3.7
	m=20 N=5	33709.0	1488.391	8.213	1480.981	125.801	3.0	1486.171	26.847	3.5
	m=20 N=10	30172.0	1252.843	8.425	1249.941	848.932	4.0	1251.237	25.363	3.1
	m=20 N=20	28366.2	1165.938	12.824	—	—	—	1163.931	33.712	3.0
	m=30 N=10	43617.4	1996.981	11.386	—	—	—	1994.218	39.241	4.1
	m=30 N=20	42890.8	1668.927	13.214	—	—	—	1667.983	36.382	3.0
	m=50 N=10	72851.2	3046.219	51.281	—	—	—	3045.831	131.128	3.7
	m=50 N=20	69339.2	2923.483	58.424	—	—	—	2919.218	194.214	3.5
Large scale	m=100 N=10	141964.2	6245.314	75.131	—	—	—	6242.501	224.609	3.2
	m=100 N=20	136463.6	5887.461	112.345	—	—	—	5881.864	323.535	3.0
	m=100 N=50	133984.8	5369.274	171.184	—	—	—	5365.305	578.214	3.4

Tab. 2 Comparisons of GA, RA-DP and RAGA on high constraint strength of CCMCKP instances

Samples		W	GA		RADP			RAGA		
			Cost	Runtime	Cost	Runtime	RA times	Cost	Runtime	RA times
Small scale	m=3 N=3	6210.8	253.616	1.813	252.944	0.139	3.0	252.944	4.435	3.0

	m=3 N=5	4919.9	288.174	2.291	285.938	0.092	3.0	285.938	5.124	3.0
	m=3 N=10	3646.8	211.919	2.311	211.919	0.012	1.0	211.919	1.932	1.0
	m=3 N=20	3812.9	282.195	2.751	281.237	0.027	3.0	281.262	7.531	3.0
	m=5 N=3	8315.3	575.102	2.319	572.690	0.022	4.0	572.690	7.918	4.0
	m=5 N=5	7629.6	409.017	2.521	408.884	0.038	7.0	409.893	13.218	6.4
	m=5 N=10	6743.0	383.772	2.741	380.126	0.031	3.0	380.126	7.092	3.0
	m=5 N=20	6070.4	312.184	2.923	311.031	0.053	3.0	311.031	8.872	3.0
Medium scale	m=10 N=3	19094.4	942.312	9.154	936.724	0.0337	3.0	936.724	16.091	3.0
	m=10 N=5	16431.4	751.231	8.472	749.653	0.152	3.0	749.653	15.891	3.0
	m=10 N=10	13278.5	779.812	8.943	778.559	0.715	3.0	779.132	21.962	3.2
	m=10 N=20	11807.6	653.816	9.248	648.780	2.153	8.0	652.532	67.142	7.6
	m=20 N=3	33404.2	1563.292	8.427	1560.820	0.891	3.0	1560.820	21.217	3.0
	m=20 N=5	30608.0	1651.176	9.693	1647.302	24.952	3.0	1649.219	34.657	3.4
	m=20 N=10	26362.0	1411.235	8.714	1400.262	938.834	4.0	1406.175	31.546	3.6
	m=20 N=20	24027.1	1255.136	9.306	—	—	—	1251.169	38.701	3.2
	m=30 N=10	37851.2	2208.684	9.238	—	—	—	2205.321	39.134	3.3
	m=30 N=20	36520.4	1759.714	18.284	—	—	—	1756.831	54.164	3.3
	m=50 N=10	63168.1	3284.115	51.453	—	—	—	3277.226	231.765	5.2
	m=50 N=20	58517.1	3144.204	61.241	—	—	—	3141.460	244.526	4.1
Large scale	m=100 N=10	122523.6	6923.245	83.429	—	—	—	6910.173	362.752	4.8
	m=100 N=20	114579.3	6461.935	134.325	—	—	—	6459.425	482.841	3.9

m=100									
N=50	110990.4	5575.325	253.842	—	—	—	5567.129	881.142	3.6

In conclusion, RAGA proves to be a highly efficient algorithm that excels in handling larger-scale problems and provides superior solutions compared to both RADP and GA. Its ability to maintain efficiency even with the increase in problem size makes it an ideal choice for practical applications involving large-scale optimization tasks.

5 Conclusion

In the course of this research, the Chance-Constrained Multi-Choice Knapsack Problem (CCMCKP) has become the central focus, given its significant applications in fields such as portfolio optimization, IT system components, and supply chain management. The practical value of solving this problem cannot be overstated.

To address the CCMCKP, this study employs a data-driven approach that leverages the Central Limit Theorem (CLT) to convert the chance constraints into deterministic risk-averse constraints, which are then solved using a Genetic Algorithm (GA). Additionally, two solution strategies are proposed: the RADP method based on dynamic programming and the RAGA method, which combines genetic algorithms and dynamic programming. While the Genetic Algorithm cannot guarantee a global optimal solution, it offers significant scalability advantages when dealing with large-scale problems. On the other hand, the RADP method is suitable only for small-scale problems. The RAGA method excels in solving large-scale problems, rapidly producing feasible solutions that, although not globally optimal, possess high practical value.

By constructing diverse test cases, the experimental results indicate that as the problem scale increases, the running time of all algorithms tends to increase. For small-scale problems, RADP is able to provide the global optimal solution, with results from RAGA being nearly identical. In the case of medium-scale problems, RAGA performs comparably to RADP on instances solvable by RADP, and as the problem scale increases further, RAGA maintains feasibility even when RADP fails. For large-scale problems, RAGA's advantages become even

more apparent. In terms of cost, the GA-based solutions are slightly inferior to those of RAGA. Furthermore, the RA times metric reveals that as the problem scale grows, RAGA demonstrates higher exploration efficiency, requiring fewer instances of the RAMCKP to explore the solution space thoroughly.

In conclusion, the CCMCKP model and the proposed solution methods presented in this paper hold both theoretical and practical significance. The RADP method guarantees exact solutions for small-scale problems, while Genetic Algorithms and RAGA offer scalability for large-scale problem solving, with RAGA exhibiting superior performance. This lays a solid foundation for future research and practical applications, with the potential for further optimization and refinement of solution approaches.

References

- [1] KELLERER H, PFERSCHY U, PISINGER D. The multiple-choice knapsack problem [M]. Knapsack Problems. Berlin: Springer, 2004: 317-347
- [2] KRZYSZTOF DUDZIŃSKI, WALUKIEWICZ S. Exact methods for the knapsack problem and its generalizations [J]. European Journal of Operational Research, 1987, 28 (1): 3-21
- [3] HIFI M, MICHRAFY M, SBIHI A. A reactive local search-based algorithm for the multiple-choice multi-dimensional knapsack problem [J]. Computational Optimization & Applications, 2006, 33 (2): 271-285
- [4] KOZANIDIS G, MELACHRINOUDIS E, SOLOMON M M. The linear multiple choice Knapsack problem [J]. Operations Research Letters, 1979, 8 (2): 95-100
- [5] DYER M E, KAYAL N, WALKER J. A branch and bound algorithm for solving the multiple-choice knapsack problem [J]. Journal of Computational & Applied Mathematics, 1984, 11 (2): 231-249
- [6] LI X, LIU S, WANG J, et al. Chance-constrained multiple-choice knapsack problem: model, algorithms, and applications [EB/ OL]. [2023-11-15]. <https://arxiv.org/pdf/2306.14690.pdf>
- [7] NASRALLAH A, THYAGATURU A S, ALHARBI Z, et al. Ultra-Low Latency (ULL) networks: the IEEE TSN and IETF DetNet standards and related 5G ULL research [J]. IEEE Communications Surveys & Tutorials, 2018, 21 (1): 88-145
- [8] LI Xuanfeng, LIU Shengcai, TANG Ke. Novel genetic algorithm for solving chance-

- constrained multiple-choice Knapsack problems [J]. Journal of Computer Applications, 2024, 44 (5): 1378-1385
- [9] ZHANG X, MA J, CHENG Z, et al. Trajectory generation by chance-constrained nonlinear MPC with probabilistic prediction [J]. IEEE Transactions on Cybernetics, 2021, 51 (7): 3616-3629
- [10] TANG K, LIU S, YANG P, et al. Few-shots parallel algorithm portfolio construction via co-evolution [J]. IEEE Transactions on Evolutionary Computation, 2021, 25 (3): 595-607